

Arrays

1. Low-Level Arrays
2. Compact Array
3. Multidimensional Arrays (Matrix)
4. Sparse Matrix

A.1 Low-Level Arrays

In this section we will look at the principles behind low-level arrays. In `C++` we get to work with the real thing: a `C++` array **is** a contiguous block of memory, and it is the foundation of dynamic sequences such as `std::vector`. Understanding it explains the performance of every sequence-based structure we will meet.

In this section we will look at the principles behind low-level arrays. In `C++` we get to work with the real thing: a `C++` array is a contiguous block of memory, and it is the foundation of dynamic sequences such as `std::vector`. Understanding it explains the performance of every sequence-based structure we will meet.

The array is a data type common to C, C++, Java, and many other programming languages; dynamic sequences such as `vector` are built on top of it.

In this section we will look at the principles behind low-level arrays. In `C++` we get to work with the real thing: a `C++` array is a contiguous block of memory, and it is the foundation of dynamic sequences such as `std::vector`. Understanding it explains the performance of every sequence-based structure we will meet.

The array is a data type common to C, C++, Java, and many other programming languages; dynamic sequences such as `vector` are built on top of it.

The array is very simple and is only capable of **storing one kind of data**. For example, you could have an array of integers or an array of floating point numbers, but you cannot mix the two in a single array.

The array only supports two operations: **indexing** and **assignment to an array index**.

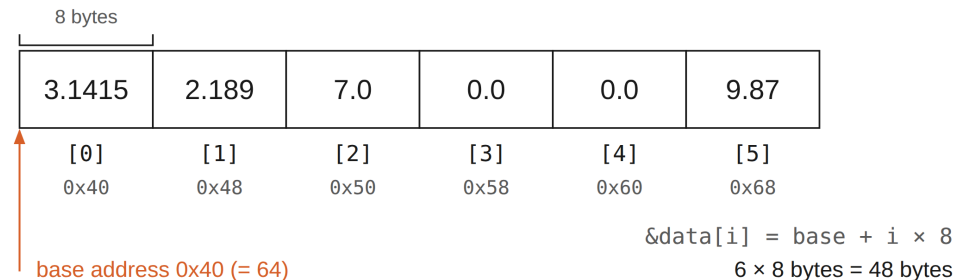
The array only supports two operations: **indexing and assignment to an array index**.

The best way to think about an array is that it is one **continuous block of bytes** in the computer's memory. This block is divided up into n -byte chunks where n is based on the data type that is stored in the array. Figure below illustrates the idea of an array that is sized to hold six floating point values.

The array only supports two operations: **indexing** and **assignment to an array index**.

The best way to think about an array is that it is one **continuous block of bytes** in the computer's memory. This block is divided up into n -byte chunks where n is based on the data type that is stored in the array. Figure below illustrates the idea of an array that is sized to hold six floating point values.

```
double data[6] = {3.1415, 2.189, 7.0, 0.0, 0.0, 9.87};
```



In `C++`, each `double` uses 8 bytes of memory, so an array of six doubles uses a total of 48 bytes. The base address is the location in memory where the array starts. The `sizeof` operator reports the size of any type or object:

In C++, each `double` uses 8 bytes of memory, so an array of six doubles uses a total of 48 bytes. The base address is the location in memory where the array starts. The `sizeof` operator reports the size of any type or object:

```
In [5]: using namespace std;
cout << sizeof(double) << " bytes per double" << endl;
cout << sizeof(int) << " bytes per int" << endl;
cout << sizeof(char) << " byte per char" << endl;

double data[6];
cout << sizeof(data) << " bytes for the whole array" << endl;
cout << &data[0] << " <- base address" << endl;
cout << &data[1] << " <- 8 bytes later" << endl;
```

```
8 bytes per double
4 bytes per int
1 byte per char
48 bytes for the whole array
0x7f3e9ffbb0 <- base address
0x7f3e9ffbb8 <- 8 bytes later
```

You can see where an object lives with the address-of operator: `cout << &x` prints something like `0x7f3e9fbb0`, the memory address of `x`. The address is very important because an array implements the index operator using a very simple calculation:

You can see where an object lives with the address-of operator: `cout << &x` prints something like `0x7f3e9fbb0`, the memory address of `x`. The address is very important because an array implements the index operator using a very simple calculation:

```
item_address = base_address + index * size_of_object
```

For example, suppose that our array starts at location `0x000040`, which is 64 in decimal. To calculate the location of the object at position 4 in the array we simply do the arithmetic:

Clearly this kind of calculation is .

For example, suppose that our array starts at location `0x000040`, which is 64 in decimal. To calculate the location of the object at position 4 in the array we simply do the arithmetic:

$$64 + 4 \times 8$$

Clearly this kind of calculation is $O(1)$.

Of course this comes with some risks:

- First, since the size of an array is fixed, one cannot just add things on to the end of the array indefinitely without some serious consequences.

For example, suppose that our array starts at location `0x000040`, which is 64 in decimal. To calculate the location of the object at position 4 in the array we simply do the arithmetic:

$$64 + 4 \times 8$$

Clearly this kind of calculation is $O(1)$.

Of course this comes with some risks:

- First, since the size of an array is fixed, one cannot just add things on to the end of the array indefinitely without some serious consequences.
- Second, in some languages, like C, the bounds of the array are not even checked, so even though your array has only six elements in it, assigning a value to index 7 will not result in a runtime error!

```
In [6]: display_quiz(path+"array1.json", max_width=800)
```

If a low-level array starts at memory address 1024 and each element (e.g., an integer) occupies 4 bytes of memory, what is the address of the element at index 8?

☐ 1056

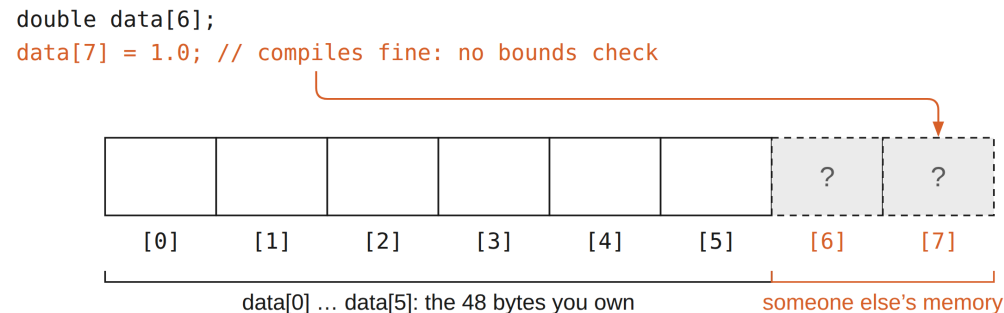
☐ 1060

☐ 1028

☐ 1032

As you might imagine this can cause big problems that are hard to track down. In the Linux operating system, accessing a value that is beyond the boundaries of an array will often produce the rather uninformative error message "segmentation fault." C++ does not check array bounds: writing past the end is undefined behavior.

As you might imagine this can cause big problems that are hard to track down. In the Linux operating system, accessing a value that is beyond the boundaries of an array will often produce the rather uninformative error message "segmentation fault." **C++** does not check array bounds: writing past the end is undefined behavior.



Undefined behavior: silent corruption, or the OS stops you with
Segmentation fault (core dumped)

The general strategy that a dynamic array (like `std::vector`) uses is as follows:

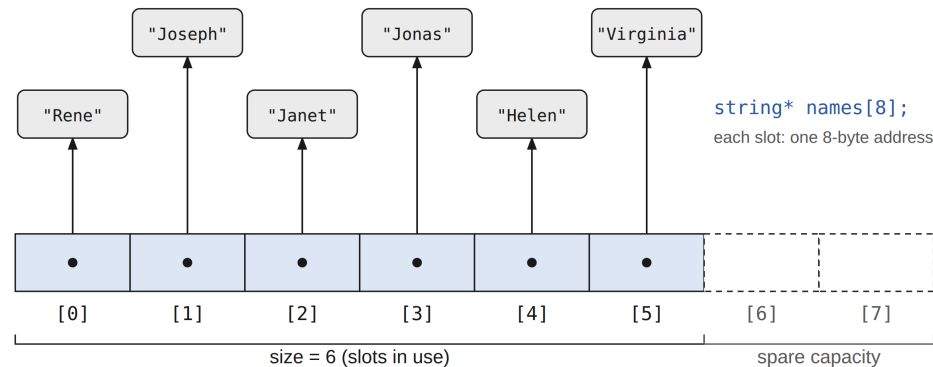
- Allocate a raw array with **extra capacity**, and remember how many slots are actually used.
- When the array fills up, allocate a **bigger** raw array (typically twice the size), copy the elements over, and free the old one. Keep indexing $O(1)$ by storing elements contiguously.

The general strategy that a dynamic array (like `std::vector`) uses is as follows:

- Allocate a raw array with **extra capacity**, and remember how many slots are actually used.
- When the array fills up, allocate a **bigger** raw array (typically twice the size), copy the elements over, and free the old one. Keep indexing $O(1)$ by storing elements contiguously.
- `std::vector` over-allocates: `capacity()` can exceed `size()`, so most `push_back()` calls need no new allocation.

The general strategy that a dynamic array (like `std::vector`) uses is as follows:

- Allocate a raw array with **extra capacity**, and remember how many slots are actually used.
- When the array fills up, allocate a **bigger** raw array (typically twice the size), copy the elements over, and free the old one. Keep indexing $O(1)$ by storing elements contiguously.
- `std::vector` over-allocates: `capacity()` can exceed `size()`, so most `push_back()` calls need no new allocation.



Let's look at how the strategy outlined above works for a very simple implementation. To begin, we will only implement the constructor and the `append()`, `size()` and `isEmpty()` methods. We will call this class `ArrayList`.

Let's look at how the strategy outlined above works for a very simple implementation. To begin, we will only implement the constructor and the `append()`, `size()` and `isEmpty()` methods. We will call this class `ArrayList`.

```
// https://github.com/phonchi/python3/blob/master/cppds/arrayList.hpp
class ArrayList {
    public:
        ArrayList(int initialCapacity = 8) {
            maxSize = initialCapacity;
            lastIndex = 0;
            myArray = new int[maxSize]; // We allocate a raw array with `new`
        }
        ~ArrayList() {
            delete[] myArray;
        }
    private:
        int maxSize;
        int lastIndex;
        int* myArray; // raw dynamic array
};
```

```
void push_back(int val) {  
    if (lastIndex == maxSize) grow();  
    myArray[lastIndex] = val;  
    lastIndex++;  
}  
int size() {  
    return lastIndex;  
}  
bool empty() {  
    return lastIndex == 0;  
}
```

Next, let us turn to the index operators. Below is our `C++` implementation for indexing — we overload `operator[]` and check the bounds ourselves, throwing an exception on a bad index:

Next, let us turn to the index operators. Below is our `C++` implementation for indexing — we overload `operator[]` and check the bounds ourselves, throwing an exception on a bad index:

The built-in index operator already hides the address calculation `base + idx * sizeof(int)`; our `operator[]` adds a bounds check and then simply forwards to it:

Next, let us turn to the index operators. Below is our C++ implementation for indexing — we overload `operator[]` and check the bounds ourselves, throwing an exception on a bad index:

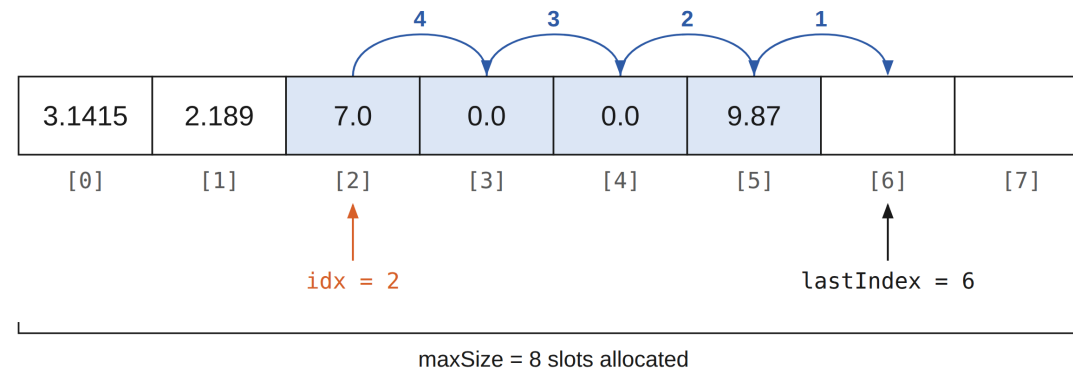
The built-in index operator already hides the address calculation `base + idx * sizeof(int)`; our `operator[]` adds a bounds check and then simply forwards to it:

```
int& operator[](int idx) {  
    if (0 <= idx && idx < lastIndex) {  
        return myArray[idx];  
    }  
    throw out_of_range("index out of bounds");  
}
```

Returning a **reference** (`int&`) makes both reading `a[i]` and assigning `a[i] = val` work with a single operator — the compiler picks the right use automatically.

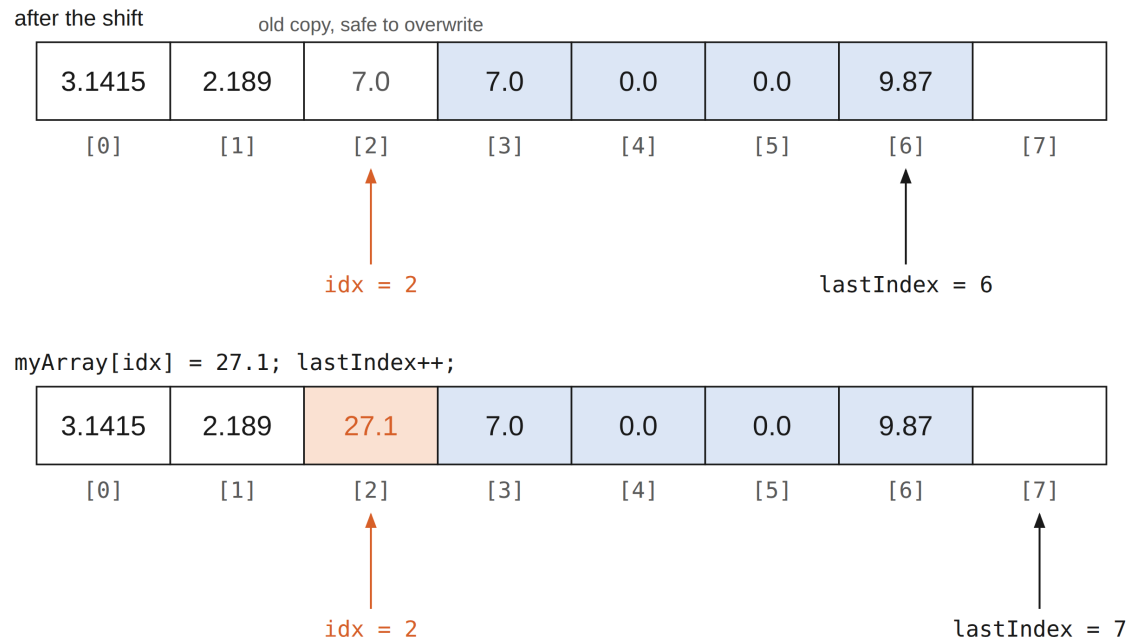
Finally, let's take a look at one of the more expensive array operations, `insert()`. When we insert an item into an `ArrayList` we will need to first shift everything in the array at the insertion point and beyond ahead by one index position in order to make room for the item we are inserting. The process is illustrated in Figure below, where `lastIndex` is the first empty slot:

Finally, let's take a look at one of the more expensive array operations, `insert()`. When we insert an item into an `ArrayList` we will need to first shift everything in the array at the insertion point and beyond ahead by one index position in order to make room for the item we are inserting. The process is illustrated in Figure below, where `lastIndex` is the first empty slot:



The key to implementing insert correctly is to realize that as you are shifting values in the array you do not want to overwrite any important data. **The way to do this is to work from the end of the array back toward the insertion point, copying data forward.**

The key to implementing insert correctly is to realize that as you are shifting values in the array you do not want to overwrite any important data. **The way to do this is to work from the end of the array back toward the insertion point, copying data forward.**



Our implementation of insert is shown below. Note how the loop starts at `lastIndex` and runs down to `idx + 1`, so we copy existing data into the unused slot first, and then subsequent values are copied over old values that have already been shifted.

Our implementation of insert is shown below. Note how the loop starts at `lastIndex` and runs down to `idx + 1`, so we copy existing data into the unused slot first, and then subsequent values are copied over old values that have already been shifted.

```
void insert(int idx, int val) {  
    for (int i = lastIndex; i > idx; i--) {  
        myArray[i] = myArray[i - 1];  
    }  
    myArray[idx] = val;  
    lastIndex++;  
}
```

Our implementation of insert is shown below. Note how the loop starts at `lastIndex` and runs down to `idx + 1`, so we copy existing data into the unused slot first, and then subsequent values are copied over old values that have already been shifted.

```
void insert(int idx, int val) {  
    for (int i = lastIndex; i > idx; i--) {  
        myArray[i] = myArray[i - 1];  
    }  
    myArray[idx] = val;  
    lastIndex++;  
}
```

If the loop had started at the insertion point and copied that value to the next larger index position in the array, the old value would have been lost forever!

The performance of the insert is since in the worst case we want to insert something at index 0 and we have to shift the entire array forward by one. On average we will only need to shift half of the array, but this is still .

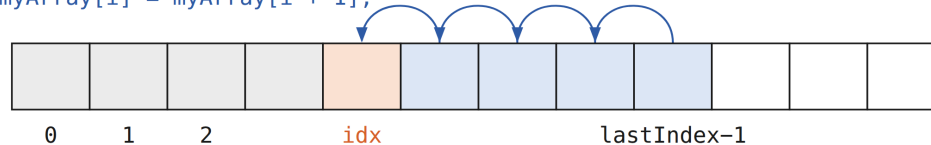
The performance of the insert is $O(n)$ since in the worst case we want to insert something at index 0 and we have to shift the entire array forward by one. On average we will only need to shift half of the array, but this is still $O(n)$.

The class offers another method, named `erase()`, that allows the caller to specify the index that should be removed. Formally, it removes only the first occurrence of such a value from an array, or throws an exception if no such value is found.

The performance of the insert is $O(n)$ since in the worst case we want to insert something at index 0 and we have to shift the entire array forward by one. On average we will only need to shift half of the array, but this is still $O(n)$.

The class offers another method, named `erase()`, that allows the caller to specify the index that should be removed. Formally, it removes only the first occurrence of such a value from an array, or throws an exception if no such value is found.

```
for (int i = idx; i < lastIndex - 1; ++i)
    myArray[i] = myArray[i + 1];
```



```
--lastIndex; // the old last slot becomes free
```

```
erase(idx): O(n) in the worst case (idx = 0)
```

```
void erase(int idx) {  
    for (int i = idx; i < lastIndex - 1; ++i)  
        myArray[i] = myArray[i + 1];  
    --lastIndex;  
}
```

```
void erase(int idx) {  
    for (int i = idx; i < lastIndex - 1; ++i)  
        myArray[i] = myArray[i + 1];  
    --lastIndex;  
}
```

It checks the index in $O(1)$ and then shifts every element after idx one position to the left, which takes $n - 1 - idx$ moves. The worst case is erasing the first element ($idx = 0$), so the complexity is again $O(n)$.

```

In [8]: #include <iostream>
#include "pythonds3/cppds/arraylist.hpp" // the complete class of this section
using namespace std;
void print(const ArrayList& a) {
    for (int i = 0; i < a.size(); i++) cout << a[i] << " ";
    cout << "(size " << a.size() << ", capacity " << a.capacity() << ")" << endl;
}
int main() {
    ArrayList myArray;
    myArray.push_back(31); myArray.push_back(77);
    myArray.push_back(17); myArray.push_back(93);
    print(myArray);
    cout << myArray[3] << " " << myArray.size() << endl;

    myArray.insert(3, 20);
    myArray.erase(0); // remove 31, which sits at index 0
    print(myArray);

    myArray.erase(2);
    myArray[0] = 50;
    print(myArray);
    return 0;
}

```

```

31 77 17 93 (size 4, capacity 8)
93 4
77 17 20 93 (size 4, capacity 8)
50 17 93 (size 3, capacity 8)

```

In [9]: `display_quiz(path+"array2.json", max_width=800)`

In C++, an array of pointers (for example, `Animal* zoo[10]`) is considered a referential array. Why?

- ☐ Because each cell stores a memory address that points to an object living elsewhere in memory.
- ☐ Because it stores the actual bits of each object directly in the array cells.
- ☐ Because it only allows elements of exactly the same size to be referenced.
- ☐ Because pointer arrays are automatically allocated on the heap.

A.2 Compact Array

Assume that we have a collection of names such as {"Rene", "Joseph", "Janet", "Jonas", "Helen", "Virginia", ...}. In a referential layout the array cells hold **pointers** to string objects living elsewhere; each cell has the same fixed size no matter how long the name is.

Assume that we have a collection of names such as {"Rene", "Joseph", "Janet", "Jonas", "Helen", "Virginia", ...}. In a referential layout the array cells hold **pointers** to string objects living elsewhere; each cell has the same fixed size no matter how long the name is.

An array requires every cell to use the same number of bytes. Yet names have different lengths, so how can an array hold them?

Assume that we have a collection of names such as {"Rene", "Joseph", "Janet", "Jonas", "Helen", "Virginia", ...}. In a referential layout the array cells hold **pointers** to string objects living elsewhere; each cell has the same fixed size no matter how long the name is.

An array requires every cell to use the same number of bytes. Yet names have different lengths, so how can an array hold them?

Instead, we store an **array of pointers** (`string*`): each slot holds a fixed-size 8-byte address of a string stored elsewhere. Although the strings differ in length, every slot has the same size, so indexing stays $O(1)$!

On the other hand, a C-style string such as `char s[] = "SAMPLE"` is a `char` array that stores the characters themselves plus a terminating `'\0'` (not an array of pointers). We will refer to this more direct representation as a compact array because the array is storing the bits that represent the primary data (characters, in the case of strings)!

On the other hand, a C-style string such as `char s[] = "SAMPLE"` is a `char` array that stores the characters themselves plus a terminating `'\0'` (not an array of pointers). We will refer to this more direct representation as a compact array because the array is storing the bits that represent the primary data (characters, in the case of strings)!

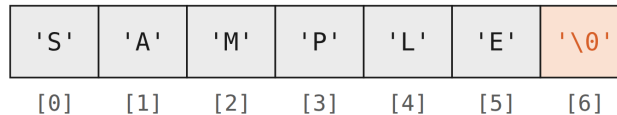
```
char s[] = "SAMPLE";
```

'S'	'A'	'M'	'P'	'L'	'E'	'\0'
[0]	[1]	[2]	[3]	[4]	[5]	[6]

7 bytes: 1 byte per char + the terminator

On the other hand, a C-style string such as `char s[] = "SAMPLE"` is a `char` array that stores the characters themselves plus a terminating `'\0'` (not an array of pointers). We will refer to this more direct representation as a compact array because the array is storing the bits that represent the primary data (characters, in the case of strings)!

```
char s[] = "SAMPLE";
```



7 bytes: 1 byte per char + the terminator

The overall memory usage will be much lower for a compact structure because there is no overhead devoted to the explicit storage of the sequence of memory references (in addition to the primary data)!

A referential structure will typically use 8 bytes (64 bits) for the memory address stored in the array, on top of whatever memory the object itself uses! In `C++`, an ordinary array **is already compact**: the element type determines exactly how many bytes each slot occupies.

A referential structure will typically use 8 bytes (64 bits) for the memory address stored in the array, on top of whatever memory the object itself uses! In `C++`, an ordinary array **is already compact**: the element type determines exactly how many bytes each slot occupies.

In [10]:

```
using namespace std;
int primes[] = {2, 3, 5, 7, 11, 13, 17, 19};
cout << sizeof(primes[0]) << " bytes per element" << endl;
cout << sizeof(primes) << " bytes in total" << endl;
cout << sizeof(primes) / sizeof(primes[0]) << " elements" << endl;

string* names[3]; // a referential array: 3 pointers, 8 bytes each
cout << sizeof(names) << " bytes for three pointers" << endl;
```

4 bytes per element

32 bytes in total

8 elements

24 bytes for three pointers

A referential structure will typically use 8 bytes (64 bits) for the memory address stored in the array, on top of whatever memory the object itself uses! In `C++`, an ordinary array is **already compact**: the element type determines exactly how many bytes each slot occupies.

In [10]:

```
using namespace std;
int primes[] = {2, 3, 5, 7, 11, 13, 17, 19};
cout << sizeof(primes[0]) << " bytes per element" << endl;
cout << sizeof(primes) << " bytes in total" << endl;
cout << sizeof(primes) / sizeof(primes[0]) << " elements" << endl;

string* names[3]; // a referential array: 3 pointers, 8 bytes each
cout << sizeof(names) << " bytes for three pointers" << endl;
```

4 bytes per element

32 bytes in total

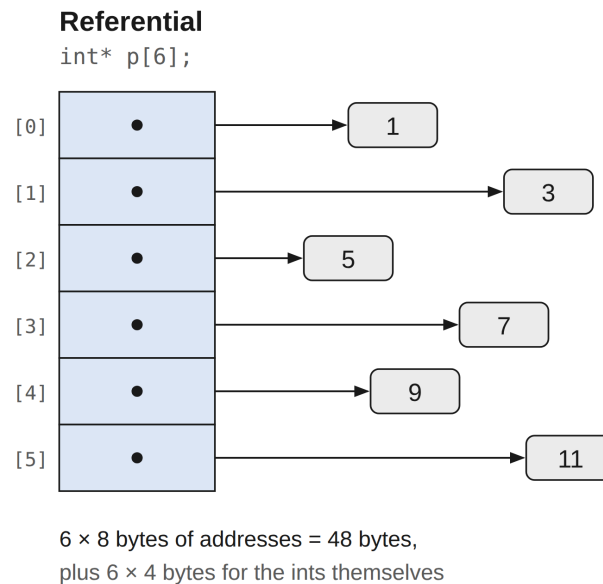
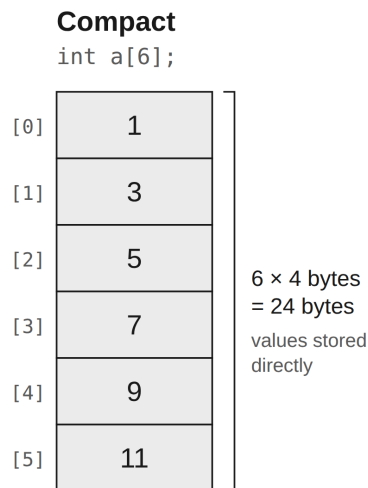
8 elements

24 bytes for three pointers

2	3	5	7	11	13	17	19	4 bytes each
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	8 × 4 = 32 bytes
+0	+4	+8	+12	+16	+20	+24	+28	byte offset

The element type tells the **compiler** exactly how many bytes each element uses: `int` (4 bytes), `double` (8 bytes), `char` (1 byte). Compare a compact `int a[6]` (24 bytes) with a referential `int* p[6]` (48 bytes of pointers, plus the ints stored elsewhere):

The element type tells the **compiler** exactly how many bytes each element uses: `int` (4 bytes), `double` (8 bytes), `char` (1 byte). Compare a compact `int a[6]` (24 bytes) with a referential `int* p[6]` (48 bytes of pointers, plus the ints stored elsewhere):



In [11]: `display_quiz(path+"array3.json", max_width=800)`

When using a C++ vector, which of the following operations has a time complexity of $O(n)$ due to the need to shift elements?

☐ `vec.push_back(value)`

☐ `vec.insert(vec.begin(), value)`

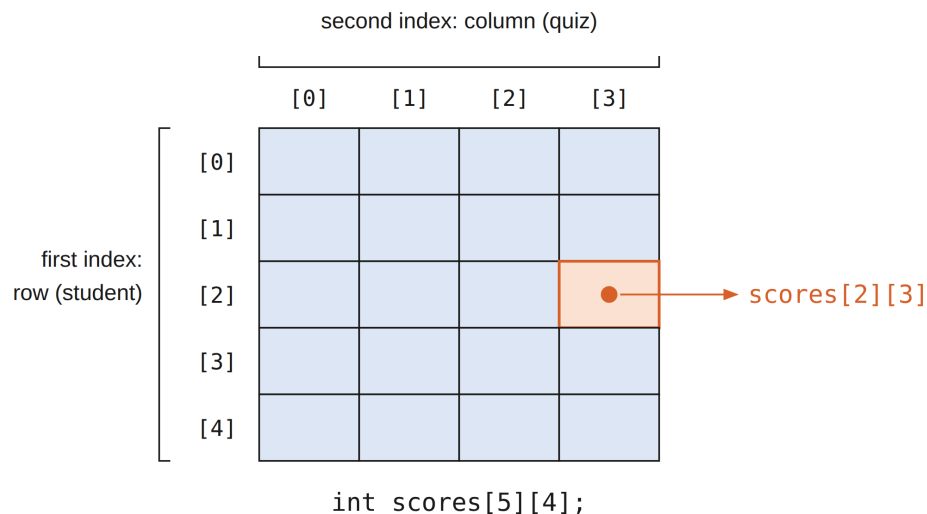
☐ `vec.pop_back()`

☐ `vec[i] = value`

A.3 Multidimensional Arrays

The arrays discussed so far are known as one-dimensional arrays because the data is organized linearly in only one direction. Many applications require that data be stored in more than one dimension. One common example is a table, which is an array that consists of rows and columns. This is commonly called a two-dimensional array (Matrix):

The arrays discussed so far are known as one-dimensional arrays because the data is organized linearly in only one direction. Many applications require that data be stored in more than one dimension. One common example is a table, which is an array that consists of rows and columns. This is commonly called a two-dimensional array (Matrix):



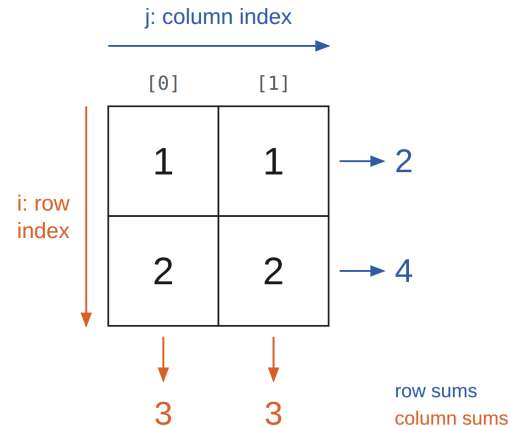
The array `int scores[5][4]` holds the scores of students in a class. There are five students (rows `[0] .. [4]`) and each student has four quiz scores (columns `[0] .. [3]`). `scores[2][3]` is the score of the third student (row 2) on the fourth quiz (column 3) — C++ indexes start at 0.

The array `int scores[5][4]` holds the scores of students in a class. There are five students (rows `[0] .. [4]`) and each student has four quiz scores (columns `[0] .. [3]`). `scores[2][3]` is the score of the third student (row 2) on the fourth quiz (column 3) — C++ indexes start at 0.

Note that multidimensional arrays — arrays with more than two dimensions — are also possible.

The array `int scores[5][4]` holds the scores of students in a class. There are five students (rows `[0] .. [4]`) and each student has four quiz scores (columns `[0] .. [3]`). `scores[2][3]` is the score of the third student (row 2) on the fourth quiz (column 3) — C++ indexes start at 0.

Note that multidimensional arrays — arrays with more than two dimensions — are also possible.



```
int M[2][2] = {{1, 1}, {2, 2}};
```

```
In [12]: using namespace std;
// a matrix: initialize a 2D array with nested braces
int M[2][2] = {{1, 1}, {2, 2}};

cout << "M has " << sizeof(M) / sizeof(M[0]) << " rows and "
      << sizeof(M[0]) / sizeof(M[0][0]) << " columns, "
      << sizeof(M) << " bytes in total" << endl;
```

M has 2 rows and 2 columns, 16 bytes in total

```
In [12]: using namespace std;
// a matrix: initialize a 2D array with nested braces
int M[2][2] = {{1, 1}, {2, 2}};

cout << "M has " << sizeof(M) / sizeof(M[0]) << " rows and "
      << sizeof(M[0]) / sizeof(M[0][0]) << " columns, "
      << sizeof(M) << " bytes in total" << endl;
```

M has 2 rows and 2 columns, 16 bytes in total

C++ arrays have no slicing: to read a row fix `i` and loop over `j`; to read a column fix `j` and loop over `i`.

```
In [16]: using namespace std;
int M[2][2] = {{1, 1}, {2, 2}};
for (int j = 0; j < 2; j++) {
    cout << M[0][j] << " ";    // row 0
}
cout << M[0] << endl;
cout << endl;
```

```
1 1 0x47d29ff650
```

In [16]:

```
using namespace std;
int M[2][2] = {{1, 1}, {2, 2}};
for (int j = 0; j < 2; j++) {
    cout << M[0][j] << " ";    // row 0
}
cout << M[0] << endl;
cout << endl;
```

1 1 0x47d29ff650

In [14]:

```
using namespace std;
int M[2][2] = {{1, 1}, {2, 2}};
for (int i = 0; i < 2; i++) {
    cout << M[i][0] << " ";    // column 0
}
cout << endl;
```

1 2

```
In [16]: using namespace std;
int M[2][2] = {{1, 1}, {2, 2}};
for (int j = 0; j < 2; j++) {
    cout << M[0][j] << " ";    // row 0
}
cout << M[0] << endl;
cout << endl;
```

1 1 0x47d29ff650

```
In [14]: using namespace std;
int M[2][2] = {{1, 1}, {2, 2}};
for (int i = 0; i < 2; i++) {
    cout << M[i][0] << " ";    // column 0
}
cout << endl;
```

1 2

```
In [17]: using namespace std;
int M[2][2] = {{1, 1}, {2, 2}};
cout << M[1][1] << endl;    // row 1, column 1
```

2

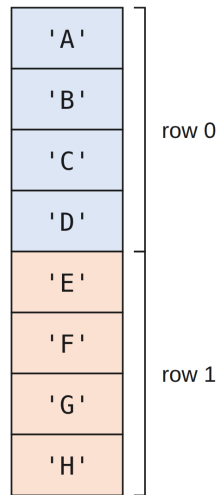
A.3.1 Memory layout

The indexes in a one-dimensional array directly define the relative positions of the element in actual memory. A two-dimensional array, however, represents rows and columns. How each element is stored in memory depends on the computer!

The indexes in a one-dimensional array directly define the relative positions of the element in actual memory. A two-dimensional array, however, represents rows and columns. How each element is stored in memory depends on the computer!

Most computers use **row-major** storage, in which an entire row of an array is stored in memory before the next row. However, a computer may store the array using **column-major** storage, in which the entire column is stored before the next column. The following figure shows a two-dimensional array and how it is stored in memory using row-major or column-major storage. Row-major storage is more common.

Row-major



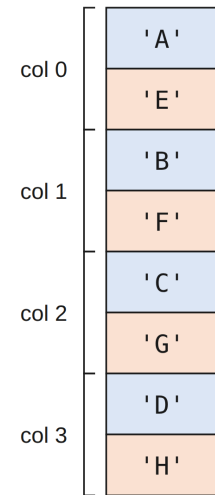
C, C++

User's view: char M[2][4]

	[0]	[1]	[2]	[3]
[0]	'A'	'B'	'C'	'D'
[1]	'E'	'F'	'G'	'H'

Memory addresses grow downward in both strips.
C++ guarantees row-major: M[0][3] is followed by M[1][0].

Column-major



Fortran

C++ two-dimensional arrays are **row-major by definition** — the language standard guarantees that `M[0][0]`, `M[0][1]`, `M[1][0]`, `M[1][1]` sit consecutively in memory. We can see it directly by printing addresses:

C++ two-dimensional arrays are **row-major by definition** — the language standard guarantees that `M[0][0]`, `M[0][1]`, `M[1][0]`, `M[1][1]` sit consecutively in memory. We can see it directly by printing addresses:

```
In [18]: using namespace std;
int M[2][2] = {{1, 1}, {2, 2}};
cout << &M[0][0] << " " << &M[0][1] << endl;    // 4 bytes apart
cout << &M[1][0] << " " << &M[1][1] << endl;    // next row follows immediately
```

0x16ea5ff690 0x16ea5ff694
0x16ea5ff698 0x16ea5ff69c

Exercise 1: We have stored the two-dimensional array `students` in the memory. The array is 100×4 (100 rows and 4 columns). Show the address of the element `students[5][3]` assuming that the element `student[0][0]` is stored in the memory location with address 0 and each element occupies only one memory location. The computer uses row-major storage.

Exercise 1: We have stored the two-dimensional array students in the memory. The array is 100×4 (100 rows and 4 columns). Show the address of the element `students[5][3]` assuming that the element `student[0][0]` is stored in the memory location with address 0 and each element occupies only one memory location. The computer uses row-major storage.

```
In [ ]: #include <cassert>
        using namespace std;

        int main() {
            int student[100][4];
            for (int i = 0; i < 100; i++)
                for (int j = 0; j < 4; j++) student[i][j] = i * 4 + j;
            int* s = &student[0][0];
            // Replace ? with your answer
            assert(student[5][3] == s[?]);
            cout << "Pass" << endl;
            return 0;
        }
```

We can use the following formula to find the location of an element, assuming each element occupies one memory location:

$$y = x + Cols \times i + j$$

where x defines the start address, $Cols$ defines the number of columns in the array, i defines the row number of the element, j defines the column number of the element, and y is the address we are looking for!

In [20]: `display_quiz(path+"array4.json", max_width=800)`

In a 2D array (Matrix) with 5 rows and 10 columns stored in Row-Major order, which formula represents the 1D index of the element at row i and column j ?



$j * 5 + i$



$i + j$



$i * 5 + j$



$i * 10 + j$

A.4 Sparse Matrix

A sparse matrix is a matrix that has a value of 0 for most elements. If the ratio of Number of Non-Zero (NNZ) elements to the size is less than 0.05, the matrix is sparse!

A sparse matrix is a matrix that has a value of 0 for most elements. If the ratio of Number of Non-Zero (NNZ) elements to the size is less than 0.05, the matrix is sparse!

Dense

store all 35 entries

0	7	0	0	0	0	6
0	7	6	3	0	4	0
0	4	3	0	0	0	0
4	2	0	0	0	0	0
0	0	0	0	3	2	4

Sparse

store only the 13 non-zeros

	7					6
	7	6	3		4	
	4	3				
4	2					
				3	2	4

NNZ = 13 of 35 entries; a missing entry means 0.

Real sparse matrices are far emptier (density well below 5%).

Storing information about all the 0 elements is inefficient, so we will assume unspecified elements to be 0. Using this scheme, sparse matrices can perform faster operations and use less memory than its corresponding dense matrix representation.

Storing information about all the 0 elements is inefficient, so we will assume unspecified elements to be 0. Using this scheme, sparse matrices can perform faster operations and use less memory than its corresponding dense matrix representation.

Different sparse formats have their strengths and weaknesses. A good starting point is looking at formats that are efficient for constructing these matrices. Typically you would start with one of these forms and then convert to another when ready to do calculations.

A.4.1 Coordinate Matrix

Perhaps the simplest sparse format to understand is the COOrdinate (COO) format. This variant uses three subarrays to store the element values and their coordinate positions.

Perhaps the simplest sparse format to understand is the COOrdinate (COO) format. This variant uses three subarrays to store the element values and their coordinate positions.

		column j						
row i	0	1	2	3	4	5	6	
0			9					
1		2						
2				1				
3					5			
4				6				
5								

COO: one triple per non-zero

[0]	[1]	[2]	[3]	[4]
int row[5]				
0	1	2	3	4

int col[5]				
2	1	3	4	3

double data[5]				
9	2	1	5	6

$k = 3$: (row 3, col 4) holds 5

The savings on memory consumption is quite substantial as the matrix size increases. Managing data in a sparse structure is a fixed cost unlike the case for dense matrices.

The savings on memory consumption is quite substantial as the matrix size increases. Managing data in a sparse structure is a fixed cost unlike the case for dense matrices.

The overhead incurred from needing to manage the subarrays is becomes negligible as data grows making it a great choice for some datasets!

A.4.2 Dictionary of Keys Matrix

Dictionary Of Keys (DOK) is very much like COO except that it uses a **map with (row, column) pairs as keys** to store coordinate-data information as key-value pairs. Our implementation uses `std::map`, a balanced search tree, so a lookup costs ; an `std::unordered_map` (hash table) would give average .

Dictionary Of Keys (DOK) is very much like COO except that it uses a **map with (row, column) pairs as keys** to store coordinate-data information as key-value pairs. Our implementation uses `std::map`, a balanced search tree, so a lookup costs $O(\log n_{nz})$; an `std::unordered_map` (hash table) would give average $O(1)$.

Use this format if you need the functionality of associative containers (`map` / `unordered_map`), but be mindful that tree or hash nodes use more memory than arrays.

Dictionary Of Keys (DOK) is very much like COO except that it uses a **map with (row, column) pairs as keys** to store coordinate-data information as key-value pairs. Our implementation uses `std::map`, a balanced search tree, so a lookup costs $O(\log n_{nz})$; an `std::unordered_map` (hash table) would give average $O(1)$.

Use this format if you need the functionality of associative containers (`map` / `unordered_map`), but be mindful that tree or hash nodes use more memory than arrays.

The `SparseMatrix` class uses a `map` whose keys are `(row, column)` pairs (`pair<size_t, size_t>`) and whose values are the non-zero elements.

```

class SparseMatrix {
//https://github.com/phonchi/python3/blob/master/cppds/sparsematrix.hpp
public:
    SparseMatrix() {}
    // read access: return the stored value, or 0 for unset positions
    double operator()(size_t i, size_t j) const {
        auto it = data.find({i, j});
        if (it != data.end()) {
            return it->second;
        }
        return 0.0;
    }
private:
    map<pair<size_t, size_t>, double> data;
};

```

```

class SparseMatrix {
    //https://github.com/phonchi/pythononds3/blob/master/cppds/sparsematrix.hpp
    public:
        SparseMatrix() {}
        // read access: return the stored value, or 0 for unset positions
        double operator()(size_t i, size_t j) const {
            auto it = data.find({i, j});
            if (it != data.end()) {
                return it->second;
            }
            return 0.0;
        }
    private:
        map<pair<size_t, size_t>, double> data;
};

// write access: inserts the position if absent
double& operator()(size_t i, size_t j) {
    return data[{i, j}];
}

size_t nnz() const {
    return data.size();    // number of non-zeros
}

```

We can easily implement relational operations as follows:

We can easily implement relational operations as follows:

```
bool operator==(const SparseMatrix& other) const {  
    return data == other.data;  
}  
  
bool operator!=(const SparseMatrix& other) const {  
    return !(*this == other);  
}
```

We can easily implement relational operations as follows:

```
bool operator==(const SparseMatrix& other) const {  
    return data == other.data;  
}  
  
bool operator!=(const SparseMatrix& other) const {  
    return !(*this == other);  
}  
  
// how sparse is the matrix, given its logical dimensions?  
double sparsity(size_t rows, size_t cols) const {  
    return 1.0 - double(data.size()) / double(rows * cols);  
}
```

The arithmetic operation is slightly more involved — every coordinate present in either operand must appear in the result:

The arithmetic operation is slightly more involved — every coordinate present in either operand must appear in the result:

```
SparseMatrix operator+(const SparseMatrix& other) const {  
    SparseMatrix result;  
    for (const auto& item : data) {  
        result.data[item.first] = item.second + other(item.first.first,  
item.first.second);  
    }  
    for (const auto& item : other.data) {  
        if (data.find(item.first) == data.end()) {  
            result.data[item.first] = item.second;  
        }  
    }  
    return result;  
}
```

Multiplication only needs to visit the non-zero entries of both operands — this is where the sparse representation really pays off:

Multiplication only needs to visit the non-zero entries of both operands — this is where the sparse representation really pays off:

```
SparseMatrix operator*(const SparseMatrix& other) const {  
    SparseMatrix result;  
    for (const auto& item1 : data) {  
        for (const auto& item2 : other.data) {  
            if (item1.first.second == item2.first.first) {  
                result(item1.first.first, item2.first.second)  
                    += item1.second * item2.second;  
            }  
        }  
    }  
    return result;  
}
```

The conversion from dense matrix can be written as follows:

The conversion from dense matrix can be written as follows:

```
void fromDenseMatrix(const vector<vector<double>>& matrix) {  
    for (size_t i = 0; i < matrix.size(); ++i) {  
        for (size_t j = 0; j < matrix[i].size(); ++j) {  
            if (matrix[i][j] != 0) {  
                data[{i, j}] = matrix[i][j];  
            }  
        }  
    }  
}
```

```

In [21]: #include <iostream>
#include "pythonds3/cppds/sparsematrix.hpp"
using namespace std;

int main() {
    vector<vector<double>> denseMatrix = {{1, 0, 0}, {0, 2, 0}, {0, 0, 3}};
    SparseMatrix sparseMatrix;
    sparseMatrix.fromDenseMatrix(denseMatrix);
    cout << sparseMatrix << endl;

    SparseMatrix matrix1({{0, 1}, 1}, {{1, 1}, 2}, {{2, 2}, 3});
    SparseMatrix matrix2({{1, 1}, 3}, {{2, 2}, 4});

    cout << matrix1 + matrix2 << endl;
    cout << matrix1 - matrix2 << endl;
    cout << matrix1 * matrix2 << endl;
    return 0;
}

```

```

(0, 0): 1  (1, 1): 2  (2, 2): 3
(0, 1): 1  (1, 1): 5  (2, 2): 7
(0, 1): 1  (1, 1): -1  (2, 2): -1
(0, 1): 3  (1, 1): 6  (2, 2): 12

```

```
In [22]: display_quiz(path+"array5.json", max_width=800)
```

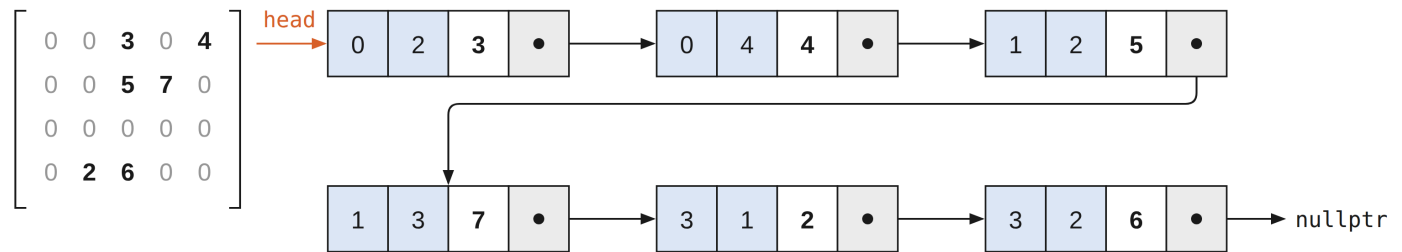
What is the primary reason for using a specialized Sparse Matrix class instead of a standard 2D array?

- ☐ To save memory and computation time when the majority of the elements in the matrix are zero.
- ☐ Because standard 2D arrays cannot have more than 100 rows.
- ☐ To make the matrix operations like addition and multiplication slower but more accurate.
- ☐ To allow the matrix to store different data types like strings and integers together.

A.4.3 Linear list

Store each non-zero as a node {row, col, value, next} in row-major order:
struct Node { int row, col; double value; Node* next; };

Store each non-zero as a node `{row, col, value, next}` in row-major order:
`struct Node { int row, col; double value; Node* next; };`



row	col	value	next
-----	-----	-------	------

```
struct Node { int row, col; double value; Node* next; };
```

```
In [23]: from jupytercards import display_flashcards  
fpath = "https://raw.githubusercontent.com/phonchi/nsysu-math208/refs/heads/main/ch3.json"  
display_flashcards(fpath)
```

Array



Next



References

1. Textbook: *Problem Solving with Algorithms and Data Structures using C++* (cppds) — <https://runestone.academy/ns/books/published/cppds/index.html>
2. Forouzan, *Foundations of Computer Science*, Ch 11 (Data Organization)

